C/C++-Kurs für Kinder ab der 5. Klasse

Jürgen ‘‘George’’ Sawinski

25. Juli 2026

Zusammenfassung

Dieser Kurs ist gedacht für Kinder ab der Grundschule mit guten mathematischen Fähigkeiten und dem Drang, programmieren zu lernen. Er entstand, nachdem die Söhne des Autors ihn über ein Jahr beknieten, ein Spiel zu entwickeln.

Der erste Teil des Kurses beschäfigt sich damit, alle grundlegenden Kenntnisse zu erwerben, um eine Konsolen-basierte Regensimulation zu in C zu programmieren, was auch mathematische und physikalische Grundlagen beinhaltet.

Der zweite Teil des Kurses wird das Ziel haben, das Spiel Tetris® in C++ umzusetzen, wobei das Hauptaugenmerk dabei auf die Grundlagen Objekt-orientierter Programmierung gelegt wird.

Inhaltsverzeichnis

1 Etwas Mathematik **2**

2 Hallo Welt! **4**

3 Zahlen **5**

4 Fibonacci **6**

A Software **7**

B Leitfaden **8**

B.1 C 8

B.1.1 Grundlagen 8

1 Etwas Mathematik

Computer rechnen im sogenannten Binärsystem, sie kennen nur 0 und 1. Um größere Zahlen darstellen zu können, werden mehrere dieser Nullen und Einsen, genannt ‘‘**Bit**’’ aneinandergereit. Am gebräuchlichsten dabei ist das ‘‘**Byte**’’ (siehe Abb. 1), das aus 8 Bit besteht.

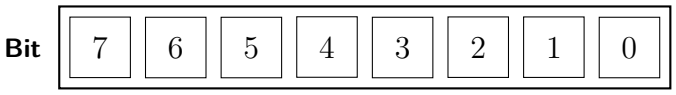


Abbildung 1: Ein Byte mit den Bit-Positionen 0 bis 7.

Jedem Bit wird nun ein Wert zugeordnet. Das erfolgt mathematisch gemäß:

$$2^b$$

wobei b die Nummer des Bits ist. Das ist eine sogenannte ‘‘Potenz’’, eine mathematische Operation die sagt, wie oft eine Zahl mit sich selbst multipliziert wird. In diesem Fall wird die ‘‘2’’ b-mal mit sich selbst multipliziert, zum Beispiel:

$$\begin{aligned} 2^1 &= 2 && = 2 \\ 2^2 &= 2 \cdot 2 && = 4 \\ 2^3 &= 2 \cdot 2 \cdot 2 && = 8 \end{aligned}$$

Interessant ist nun die Frage, was 2^0 ist. Wenn ihr genau hinschaut, dann merkt ihr, dass, um in die nächste Zeile zu kommen, ihr das Ergebnis mit 2 multipliziern müsst. Umgekehrt genauso: um in die vorhergehende Zeile zu kommen, muss man durch 2 teilen. Wenn also 2^1 zwei ist und man, um zu 2^0 zu kommen durch zwei Teilen muss, dann ergibt sich

$$2^0 = 1\,.$$

Übrigens, jede Zahl hoch 0 hat als Ergebnis die 1 (wir kommen später darauf zurück).

Um auf das **Byte** zurückzukommen, heißt dass, dass den **Bits** jeweils die Werte 1 bis 128 zugeordnet sind (siehe Abb. 2).

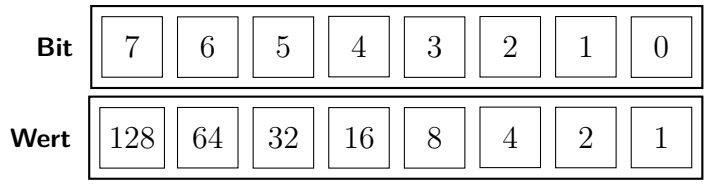


Abbildung 2: Ein Byte mit Bit-Positionen und den zugehörigen Werten (2^b).

Damit können wir nun die Zahlen 0 bis 255 darstellen, z.B.:

$$\begin{aligned} &00000000 = 0\,, \\ &00000011 = 3\,, \\ &01011000 = 88\,, \\ &11111111 = 255\,. \end{aligned}$$

Das addieren funktioniert nun genauso, wie ihr es aus der Schule, in der ihr natürlich das Zehnersystem gelernt habt. Beispiel:

```
  00100111        39
+ 00000110      +  6
      11          1      (Übertrag)
 ---------       ---
  00101101        45
```

Entsprechend lässt sich auch die Multiplikation durchführen:

```
00001010·00101010 =      10·42

 00000000
  00000000
   00000000
    00000000
     00101010
      00000000
       00101010
        00000000
         1 1                (Übertrag)
---------------
       110100100           420
```

Wenn ihr genau hinschaut, dann seht ihr, dass wir zwei Bytes multipliziert haben. Aber das Ergebnis benötigt ein Bit mehr. Das führt zu dem Problem, dass, wenn ihr auf einem Computer diese Multiplikation durchführt, das höchste Bit (die letzte ‘‘1’’ links) abgeschnitten wird und das Ergebnis 164 ist. Das gleich passiert bei einem Byte, wenn ihr zu 255 die 1 dazuzählt, das Ergebnis ist 0!

Computer können auch negative Zahlen darstellen. Dazu nützt man aus, dass, wenn 255 + 1 zum Ergebnis 0 hat, dann hat 0 − 1 als Ergebnis −1, binär also 11111111. Dies nennt man das ‘‘Einserkomplement’’ (den Begriff müsst ihr euch natürlich nicht merken). Wichtig ist: dass höchste Bit zeigt, dass es sich um eine negative Zahl handelt. Damit können dann mit einem Byte die Zahlen -128 bis 127 dargestellt werden:

```
10000000        −128
10000001        −127
...
11111111        −   1
00000000            0
00000001            1
...
01111111          127
```

Wie wird nun aus einer positiven Zahl eine Negative? Das passiert, indem man alle Bits invertiert (das heißt, aus 0 wird 1 und aus 1 eine 0) und 1 addiert:

```
10000001  ->  01111110
             +00000001
              --------
              01111111
```

2 Hallo Welt!

In alter Tradition, ist das erste Programm, das man zu sehen bekommt, gewöhnlich das ‘‘Hallo Welt!’’ Programm [1]:

```c
#include <stdio.h>

int main() {
  printf("Hallo Welt!\n");
  return 0;
}
```

3 Zahlen

```c
#include <stdio.h>

int main() {
  int zahl = 42;
  printf("Die Zahl: %d\n", zahl);
  return 0;
}
```

4 Fibonacci

```c
#include <stdio.h>

int main() {
  int a = 0;
  int b = 1;

  for(int i = 1; i <= 16; i = i+1) {
    int next = a + b;
    a = b;
    b = next;

    printf("Fibonacci %d: %d\n", i, a);
  }
  return 0;
}
```

A Software

Als Programmierumgebung wurde VSCode

B Leitfaden

B.1 C

B.1.1 Grundlagen

Datentypen

Ganzzahlen

Fließkommazahlen

Strukturen

Schleifen

Funktionen

Schleifen

Referenz

printf

Syntax:

```
#include <stdio.h>
int printf(const char *format, ...);
```

Beschreibung:
printf () schreibt formattierten Text in die Standardausgabe (stdout). Der format-String enthält:

- Normale Zeichen (werden direkt ausgegeben)
- Format-Specifiers (beginnen mit %)

Die wichtigsten Format-Specifiers:

%d	Ganzzahl (dezimal)
%f	Gleitkommazahl
%c	Zeichen (char)
%s	Zeichenkette (String)
%x	Hexadezimal (klein)
%X	Hexadezimal (groß)
%p	Zeiger (Pointer)

Beispiel:

```c
#include <stdio.h>

int main() {
    int zahl = 42;
    printf("Die Zahl ist: %d\n", zahl);
    printf("Adresse: %p\n", (void*)&zahl);
    return 0;
}

```

Rückgabewert: Die Anzahl der ausgegebenen Zeichen, oder −1 bei Fehler.

Siehe auch: scanf(), sprintf (), fprintf ()

Literatur

[1] Wikipedia. Hallo-Welt-Programm, 2003.